

Hardware-Backed TLS for IoT: Integration of SE050 Secure Element with PSA Crypto and mbedTLS

De Biasi Alessandro

12448269

1 Abstract

The increasing deployment of embedded systems in security-critical IoT environments exposes software-only cryptographic implementations to key extraction and side-channel attacks. Hardware secure elements, such as the NXP SE050, address these risks by storing private keys and executing cryptographic operations in a tamper-resistant environment.

This project implements a PSA Crypto-compatible wrapper over the existing SE05X driver on the Arduino Portenta C33 platform. The wrapper maps PSA key management and cryptographic operations (random number generation, ECDSA/RSA signing and verification, SHA-256 hashing) to the underlying SE050 hardware. The implementation is integrated with mbed TLS using the MBEDTLS_USE_PSA_CRYPT configuration, enabling a standard TLS stack to offload sensitive operations to the secure element without application-level changes.

The system is validated through a practical MQTT-over-TLS demonstration. The Portenta C33 client establishes a TLS 1.2 session with a Mosquitto broker, using the SE050 for hardware-backed random number generation and long-term key storage. Stability is verified through a four-hour continuous test with 481 message exchanges. Limitations include missing hardware support for AES-GCM on the SE050C2 variant (making TLS symmetric encryption software-only) and incomplete integration of hardware accelerated ECDH due to mbed TLS internal structure.

The complete source code and test logs are available in the project GitHub repository.

2 Affidavit

I hereby declare in lieu of an oath that:

- the submitted paper is entirely my own work and that no auxiliary materials have been used other than those indicated,
- I have fully disclosed all assistance received from third parties during the process of writing the thesis, including any significant advice from supervisors,
- any contents taken from the works of third parties or my own works that have been included either literally or in spirit have been appropriately marked and the respective source of the information has been clearly identified with precise bibliographical references (e.g. in footnotes),
- I have fully and truthfully declared the use of generative models (Artificial Intelligence, e.g. ChatGPT, Grammarly Go, Midjourney) including the product version,
- to date, I have not submitted this paper to an examining authority either in Austria or abroad and that

I am aware that a declaration contrary to the facts will have legal consequences.

Alessandro De Biasi m.p.

Klagenfurt am Wörthersee, 26/05/2026

Contents

1	Abstract	1
2	Affidavit	2
3	Introduction	5
4	Background	6
4.1	Transport Layer Security	6
4.2	Arm PSA Crypto	6
4.2.1	Opaque Keys and Key Management	6
4.2.2	Hardware and Software Backends	7
4.3	NXP SE050 Secure Element	7
4.4	Gap Between Low-Level Drivers and Standard APIs	8
4.5	Development Environment and Framework Selection	9
4.6	Motivation of the Project	9
5	Project Execution	11
5.1	Phase 1: Planning and Research	11
5.2	Phase 2: PSA Crypto Driver Integration	11
5.3	Phase 3: Secure Communication Demonstration	12
5.4	Phase 4: Final Documentation	12
6	Existing Driver Analysis	13
6.1	Driver Architecture and Design	13
6.1.1	Communication Layer	13
6.1.2	Object Model	13
6.1.3	API Organization	14
6.2	Capabilities and Features	15
6.2.1	Supported Operations	16
6.2.2	Design Strengths	16
6.3	Identified Limitations	17
6.3.1	API Design Limitations	17
6.3.2	Operational Issues	18
6.3.3	Missing Features	18
6.4	Implications for PSA Integration	18
6.5	Analysis Conclusions	19
7	Project Implementation	20
7.1	System and Software Architecture	20
7.2	PSA Crypto Hardware Driver Design	21
7.2.1	Key Management and Opaque Keys	21

7.2.2	Implemented Cryptographic Operations	22
7.3	Integration with mbedTLS	23
7.3.1	Configuration and Build System	23
7.3.2	Initialization Sequence	24
7.3.3	TLS Handshake and Cryptographic Operations	24
7.4	ECDH Key Agreement and Integration Challenges	26
7.4.1	Integration Challenges with mbedTLS	26
7.4.2	Attempted Integration	27
7.4.3	Practical Impact	27
7.5	TLS-secured MQTT Demonstration	27
7.5.1	System Architecture	28
7.5.2	Certificate Infrastructure	29
7.5.3	Connection Establishment Sequence	29
7.5.4	Message Flow and User Interaction	30
7.5.5	Hardware-Backed Security Features	31
7.5.6	Operational Validation	32
8	Project Evaluation	33
8.1	Achieved Objectives	33
8.2	Performance and Practical Observations	33
8.3	Limitations and Open Issues	34
8.3.1	SE050 Hardware Constraints	34
8.3.2	Limited Hardware Offloading	34
8.3.3	Driver Maturity	35
8.4	Security Considerations	35
8.5	Future Work	35
9	Conclusion	36
9.1	Social, Ethical, and Legal Aspects	36

3 Introduction

The use of embedded systems is increasing in security-critical environments like IoT devices, industrial systems and edge computing. Aside from industrial systems and edge computing, and as highlighted by Atzori et al. [3, p. 2801], IoT systems pose distinct challenges to the design of secure systems, as many IoT devices are resource-constrained and often deployed in an uncontrolled and dangerous environments. Because of the constraints, software solutions are often unsatisfactory because they might offer no sufficient protections causing sensitive key material to be exposed.

As a result, modern embedded systems are adopting a secure hardware component, such as secure elements. These devices allow sensitive operations to be performed in a secure environment. Concurrently, Arm Platform Security Architecture (PSA) Crypto and mbedTLS [2, 5] have come to the market to offer standard interfaces for secure hardware crypto. These frameworks help to divide the applications from their specific hardware or software implementations, making it easier to develop portable and secure applications across platforms.

This project objective is to extend the work done in the SE05X [4] driver that implements an interface to SE05X specific crypto-hardware operations by make usable with these frameworks, so, rather than focusing on the low-level communication with the hardware, the project goal is a standard-compliant cryptographic abstract interface. This abstraction allows sensitive operations to be transparently offloaded to the secure element, ensuring that private key material never leaves the protected hardware environment.

The implementation is carried out on the Arduino Portenta C33 board and is validated through a practical use case involving TLS-secured MQTT communication.

4 Background

4.1 Transport Layer Security

Transport Layer Security (TLS) is the standard protocol for ensuring secure network communications, offering confidentiality, data integrity, and endpoint authentication. Its widespread adoption across diverse application domains has established TLS as a foundational component for secure communication, particularly in embedded and Internet of Things (IoT) systems.

In IoT deployments, TLS is typically used in conjunction with lightweight application-layer protocols such as MQTT [6], which is specifically designed for constrained devices and bandwidth-limited networks. The integration of MQTT with TLS facilitates secure publish-subscribe messaging while imposing minimal additional protocol overhead.

On embedded devices, TLS is commonly realized through software libraries like mbedTLS or wolfSSL, which deliver complete TLS protocol functionality along with an extensive set of cryptographic primitives tailored for resource-limited microcontrollers [5, 8]. Although these libraries aim to reduce memory usage and computational overhead, they still depend on software-implemented cryptographic operations and require that private key material be present in main memory. As a result, sensitive keys may be exposed, as previously discussed in the context of physical and side-channel attacks.

These limitations motivate the integration of hardware-backed cryptographic mechanisms within the TLS stack, allowing sensitive operations to be offloaded to secure hardware while preserving compatibility with standard TLS protocols and application programming interfaces.

4.2 Arm PSA Crypto

The PSA Crypto API [2] provides a standardized interface for cryptographic operations, aiming to decouple application code from the underlying cryptographic implementation. It defines a uniform API for symmetric and asymmetric cryptography, key management, and secure random number generation, thereby enabling application portability across diverse hardware platforms.

4.2.1 Opaque Keys and Key Management

A foundational concept in PSA Crypto is that of *opaque keys*, that are keys that are never exposed in main memory. All operations involving such keys are executed within a trusted execution environment or dedicated secure hardware. This design substantially reduces the risk of key compromise and ensures that sensitive material remains protected even on potentially compromised systems.

Additionally, PSA Crypto standardizes key attributes and policies, enabling fine-grained control over key usage. Keys may reside in either software or hardware back-ends, and the same API is employed irrespective of the underlying implementation.

4.2.2 Hardware and Software Backends

PSA Crypto supports multiple back-end implementations:

- **Software back-end:** Implements all cryptographic operations in software. This approach is useful for rapid prototyping or for platforms lacking secure hardware.
- **Hardware back-end:** Provides integration with secure elements, hardware accelerators, or Trusted Execution Environments. The API ensures uniform interaction with these devices while keeping sensitive key material isolated from the application.

MbedTLS includes hooks that enable PSA Crypto to utilize hardware-backed implementations. This capability allows cryptographic operations to be offloaded to devices such as the NXP SE050 secure element while maintaining a consistent API at the application layer.

4.3 NXP SE050 Secure Element

The NXP SE050 is a secure element designed to provide hardware-based trust anchoring for IoT and embedded systems [7]. It facilitates the secure storage of credentials and the execution of cryptographic operations essential for security-critical communication.

It can also generate and secure random number generation. Cryptographic keys can be generated and stored internally, with configurable usage policies that prevent key extraction and restrict permitted operations, thereby enabling a security model in which private key material never leaves the secure element.

From a security assurance perspective, the SE050 is certified up to Common Criteria EAL 6+ at the operating system level and incorporates countermeasures against both invasive and non-invasive physical attacks [7, p. 3-4]. These attributes render it suitable for deployment in scenarios where devices may be physically accessible to an attacker, such as IoT nodes and edge devices.

As previously noted, prior work at Alpen-Adria-Universität Klagenfurt produced an SE05X driver [4] that provides low-level access to a broad set of SE050 functionalities, including symmetric encryption, ECC-based and RSA-based asymmetric operations, digital signing, key management, and access to the True Random Number Generator (TRNG) for secure random generation [7, p. 4-6]. While this driver enables practical use of the secure element, it exposes a hardware-specific interface and lacks integration with standardized cryptographic APIs such as those discussed above, thereby limiting its direct usability within modern TLS stacks and application frameworks.

Consequently, TLS implementations relying on PSA Crypto cannot directly leverage the SE050 without additional integration layers. Addressing this limitation by introducing a PSA-compatible abstraction layer over the existing SE05X driver is a central objective of this project.

Note

It is important to note that the secure element integrated into the Portenta C33 is the **SE050C2**. This distinction is significant because certain features listed in the SE050 data-sheet are available only in higher variants of the device, such as the **SE050E**. Examples of such features include:

- Secure I²C communication
- Support for AES-GCM and additional ECC functions

The absence of hardware support for **AES-GCM** limits the cryptographic capabilities accessible through the secure element. This limitation is particularly relevant for TLS implementations, as AES-GCM is one of the primary authenticated encryption (AEAD) cipher modes employed in modern TLS cipher suites, including those defined for TLS 1.2 and TLS 1.3. While CBC modes remain supported in TLS 1.2, their use is discouraged; in TLS 1.3, they are entirely deprecated.

4.4 Gap Between Low-Level Drivers and Standard APIs

As previously noted, although the existing SE05X driver [4] provides functional access to the SE050 secure element, it exposes a hardware-specific interface tightly coupled to the underlying device. Applications and protocol stacks that depend on standardized cryptographic abstractions cannot directly utilize this driver without substantial modification.

This gap introduces several challenges:

1. Developers must implement custom integration layers for each application, increasing overall complexity and the likelihood of security vulnerabilities.
2. The lack of standardization hinders application portability across different platforms or secure elements.
3. Software-only TLS stacks cannot transparently offload cryptographic operations to the SE050, thereby limiting the benefits of hardware-backed key protection, such as reduced attack surface and resistance to side-channel attacks.

To address this gap, a PSA-compatible wrapper over the existing SE05X driver is required, enabling seamless integration of the SE050 within modern TLS-based applications. Such an approach would allow sensitive cryptographic operations to remain hardware-protected while preserving compatibility with standard APIs.

4.5 Development Environment and Framework Selection

For the implementation of this project, the choice of development environment and software framework for the Portenta C33 platform required careful consideration. Two prominent candidates for embedded IoT development are MbedOS and Zephyr RTOS. Both provide hardware abstraction layers, device drivers, and support for real-time and networked applications, but differ in maturity, ecosystem, and integration with cryptographic frameworks.

Initially MbedOS was chosen due to several favourable characteristics (such as comprehensive official documentation, seamless integration with mbedTLS, and established support for the Arduino ecosystem) and support to the Arduino Portenta H7 board. However after changing to the Portenta C33 board it became apparent that, despite being part of the Arduino Pro family, does not have official MbedOS support. The board is based on the Renesas RA6M5 microcontroller, which is not among the officially supported MbedOS targets.

The project was then implemented using a bare-metal approach with the Arduino framework and PlatformIO build system. This approach provided several practical advantages:

- **Direct Hardware Access:** The Arduino framework for Portenta C33 provides stable drivers for WiFi, I2C, and other peripherals without the overhead of a full RTOS.
- **Simplified Deployment:** The bare-metal approach reduces complexity and eliminates RTOS-specific configuration requirements, allowing the project to focus on cryptographic integration rather than operating system internals.
- **PlatformIO Ecosystem:** PlatformIO provides professional build tooling, library management, and cross-platform development support, compensating for the absence of an RTOS while maintaining a structured development workflow.

While a full RTOS such as MbedOS or Zephyr would have provided additional features, these capabilities were not essential for demonstrating hardware-backed cryptographic operations and TLS integration. The bare-metal Arduino framework is sufficient for the project objectives, providing access to the required cryptographic libraries and hardware interfaces while maintaining a straightforward development and debugging process.

4.6 Motivation of the Project

The main motivation for this project is brought by the increasing need for secure communication in embedded and IoT systems, where traditional software-based cryptography may not provide sufficient protection against key leakage, side-channel attacks, or performance constraints.

This project aims to address this gap by providing a standardized, PSA-compatible abstraction layer over the SE05X driver. By doing so, applications can leverage hardware-backed cryptography without significant modifications to higher-level software, enabling secure TLS communications with minimal development effort.

Additionally, the project offers the opportunity to evaluate the practical implications of hardware-backed cryptography in embedded systems, including performance, usability, and security benefits. The combination of a well-defined technical problem, relevance to real-world IoT scenarios, and the chance to explore advanced embedded security techniques makes this project both challenging and practically meaningful.

5 Project Execution

The project will be structured into the following phases. Each phase includes specific sub-tasks and deliverables.

5.1 Phase 1: Planning and Research

This phase covers the initial material and background information research and the planning for the next steps. It is divided into:

Sub-Phase 1: Research and Environment Setup

This sub-phase covers the initial setup of the development environment, analysis of the existing SE05X driver, and study of software frameworks. Specifically, it includes:

- Investigation of MbedOS and Zephyr (Zephyr seems more modern and widely used, but MbedOS offers better documentation and tutorial support for our use case).
- Study of mbedTLS, which will serve as the TLS abstraction library.
- Understanding the PSA Crypto standard and its integration with secure elements.
- Selection of the MQTT/TLS software stack.

Sub-Phase 2: Secure Element and Driver Analysis

Validation of SE050 cryptographic capabilities using the existing SE05X driver as a baseline. This includes:

- Testing of key generation, signing, verification, and encryption primitives.
- Verification of secure key storage and access constraints.
- Analysis of the driver's capabilities and limitations for integration with PSA Crypto.

5.2 Phase 2: PSA Crypto Driver Integration

Design and implementation of a PSA-compatible wrapper for the SE050 over the existing SE05X driver. Key tasks include:

- Mapping PSA key attributes and policies to secure element objects.
- Enforcing key opacity and usage restrictions.
- Integration of the wrapper with mbedTLS through the PSA Crypto layer.

5.3 Phase 3: Secure Communication Demonstration

Development of a practical demonstration using the Portenta board:

- Implementation of an MQTT client example running on the board.
- TLS client authentication using keys stored in the SE050 secure element.
- End-to-end encrypted and authenticated communication with a MQTT broker.

5.4 Phase 4: Final Documentation

Preparation of the final report for the large project. Documentation work will mostly be performed in parallel with the development and evaluation phases.

6 Existing Driver Analysis

Before implementing the PSA Crypto wrapper, the existing SE05X driver [4] was analysed to understand its architecture, capabilities, and limitations. This analysis informed the design decisions for the PSA wrapper layer and identified potential integration challenges.

6.1 Driver Architecture and Design

The SE05X driver is implemented as an Arduino-compatible C++ library that provides direct access to the NXP SE050 secure element through the I2C communication interface. The driver abstracts the low-level communication protocol and APDU (Application Protocol Data Unit) construction required to interact with the SE050, presenting a higher-level API organized around cryptographic operations rather than raw hardware commands.

6.1.1 Communication Layer

The SE050 secure element communicates with the host microcontroller over I2C (Inter-Integrated Circuit) bus using a proprietary protocol based on APDUs. The driver implements this protocol stack in the `SE05XClass`, which manages:

- **I2C Session Establishment:** Initialization of the I2C bus and verification of SE050 presence through address probing and handshake sequences.
- **APDU Construction:** Building command APDUs with appropriate class bytes (CLA), instruction codes (INS), parameters (P1, P2), and data payloads according to SE050 command specifications.
- **Response Parsing:** Extracting status words and response data from APDU responses, translating SE050-specific status codes into driver return values.
- **Session Management:** Maintaining communication session state and handling session-level security if configured.

The driver uses the Arduino `Wire` library for I2C communication, making it portable across Arduino-compatible platforms that support the standard Wire API.

6.1.2 Object Model

The SE050 organizes cryptographic material and data as *secure objects* stored in non-volatile memory within the secure element. Each object is identified by a unique 32-bit object identifier and has associated metadata including:

- **Object Type:** Binary data, AES key, RSA key pair, ECC key pair, certificate, etc.

- **Access Policies:** Rules governing which operations are permitted on the object
- **Persistence:** Whether the object survives power cycles (persistent vs. transient)

The SE05X driver exposes this object model through functions that create, read, update, and delete objects. Object identifiers are managed by the application, with the driver providing helper functions to check object existence (`existsBinaryObject()`) and enumerate stored objects.

6.1.3 API Organization

The driver API is organized functionally around cryptographic primitives rather than following a specific standard such as PSA Crypto or PKCS#11. Major functional categories include:

Key Generation and Management Functions for generating cryptographic keys directly within the SE050:

- `generatePrivateKey(int keyID, byte* publicKey)` - Generate ECC P-256 key pair
- `generatePrivateRSAKey(int keyID, ...)` - Generate RSA key pair with configurable size
- `writeAESKey(int keyID, const byte* key, size_t keyLen)` - Import AES key
- `importPublicKey(int keyID, const byte* key, size_t keyLen)` - Import ECC/RSA public key

Keys remain within the secure element after generation, with only public key components exportable.

Asymmetric Cryptography

- `Sign(int keyID, const byte* hash, size_t hashLen, ...)` - ECDSA signing with DER-encoded output
- `Verify(int keyID, const byte* hash, size_t hashLen, ...)` - ECDSA signature verification
- `SignRSA(...)` - RSA signature generation (PKCS#1 v1.5)
- `VerifyRSASSA_PKCS1(...)` - RSA signature verification (PKCS#1 v1.5)
- `VerifyRSASSA_PSS(...)` - RSA-PSS signature verification

Signature operations accept pre-computed hash values rather than raw messages, following the standard practice for hardware security modules where hashing may be performed by the host.

Symmetric Cryptography

- `AES_ECB_encrypt(int keyID, const byte* input, size_t inputLen, ...)` - AES encryption in ECB mode
- `AES_ECB_decrypt(int keyID, const byte* input, size_t inputLen, ...)` - AES decryption in ECB mode

The driver provides only ECB mode for AES operations. Other modes such as CBC, CTR, or GCM are not exposed, despite the SE050 hardware supporting CBC and CTR modes. This represents a significant limitation of the current driver implementation.

Hashing

- `SHA256(const byte* input, size_t inputLen, byte* hash, ...)` - Compute SHA-256 hash in hardware

Hash computation can be performed either as a single-shot operation (as exposed by the driver) or incrementally using multi-part operations (not currently exposed).

Random Number Generation

- `random(byte* output, size_t outputLen)` - Generate random bytes using SE050 TRNG

This function provides direct access to the SE050's certified True Random Number Generator, which is critical for cryptographic applications requiring high-quality entropy.

Utility Functions

- `begin()` - Initialize SE050 communication and verify hardware availability
- `serialNumber(byte* sn, size_t snLen)` - Retrieve SE050 unique identifier
- `deleteBinaryObject(int objectID)` - Delete a stored object
- `deleteAllObjects()` - Erase all user-created objects

6.2 Capabilities and Features

The SE05X driver successfully exposes the majority of SE050 cryptographic capabilities that are relevant for embedded applications:

6.2.1 Supported Operations

- **ECC Operations:** Full support for NIST P-256 (secp256r1) elliptic curve, including key generation, ECDSA signing, and verification. Public key export is supported in DER format.
- **RSA Operations:** Support for RSA-2048 key generation and PKCS#1 v1.5 signature operations. RSA-PSS encoding and verification functions are provided but exhibit limitations (discussed below).
- **AES Encryption:** AES-128 encryption and decryption in ECB mode. Keys can be generated randomly by the host and imported, or generated directly in the SE050 if integrated with the random number generator.
- **Hash Functions:** Hardware-accelerated SHA-256 computation.
- **Random Number Generation:** Access to SE050 TRNG for entropy generation.
- **Key Storage:** Persistent storage of cryptographic keys in SE050 non-volatile memory with configurable object identifiers.

6.2.2 Design Strengths

Several aspects of the driver design facilitate integration and ease of use:

Arduino Compatibility The driver is designed as an Arduino library with standard directory structure (`src/`, `examples/`), making it straightforward to integrate into Arduino-based projects using either the Arduino IDE or PlatformIO build system. Dependencies are limited to the standard `Wire` library for I2C communication.

Clear API Semantics Function names clearly indicate the operation being performed, and return values consistently use integer status codes (1 for success, 0 for failure) rather than error enumerations. While this simplicity aids initial development, it limits error diagnostics compared to more detailed error reporting schemes.

Direct Hardware Mapping The API provides relatively direct access to SE050 capabilities without excessive abstraction layers. This allows developers who understand the SE050 architecture to utilize advanced features and optimize operations.

Object Lifetime Management The driver correctly handles object persistence, with objects surviving power cycles unless explicitly deleted. This is essential for applications requiring long-term key storage.

6.3 Identified Limitations

Despite its functional capabilities, the driver exhibits several limitations that impact its suitability for integration with standard cryptographic frameworks:

6.3.1 API Design Limitations

Lack of Standardization The driver does not conform to any established cryptographic API standard, this creates integration challenges when attempting to use the SE050 with TLS libraries that expect standardized cryptographic backends. Each operation requires explicit SE05X-specific function calls, preventing transparent hardware offloading.

Limited Error Reporting Functions return simple integer success/failure codes without detailed error information. When an operation fails, the driver provides no indication of whether the failure was due to invalid parameters, I2C communication errors, SE050 internal errors, or unsupported operations. This complicates debugging and error handling in higher-level code.

Incomplete Symmetric Cipher Support Only AES-ECB mode is exposed, despite the SE050 supporting AES-CBC and AES-CTR modes. ECB mode is generally considered insecure for most applications due to its deterministic nature and lack of diffusion across blocks. The absence of CBC and CTR modes limits the driver's utility for applications requiring secure symmetric encryption.

Furthermore, no AEAD (Authenticated Encryption with Associated Data) modes are available, which is a hardware limitation of the SE050 itself rather than a driver issue. However, the driver does not expose alternative approaches such as combining AES-CTR with HMAC for authenticated encryption.

No Multi-Part Operations The driver exposes only single-shot operations for hashing and encryption. Multi-part APIs, which allow incremental processing of large datasets, are not available despite the SE050 supporting them. This forces applications to buffer entire messages in memory before processing, which may be impractical for memory-constrained embedded systems processing large files or streams.

RSA Padding Schemes While RSA signing functions are provided, the handling of padding schemes (PKCS#1 v1.5 vs. PSS) is inconsistent. The `emsa_pss_encode()` function appears to perform PSS encoding but does not integrate cleanly with the signing flow, requiring manual encoding followed by raw RSA operations. This complexity is not well-documented in the driver.

6.3.2 Operational Issues

During integration testing, several operational issues were encountered:

AES Operation Failures Despite the SE050 hardware supporting AES-ECB operations, calls to `AES_ECB_encrypt()` and `AES_ECB_decrypt()` were consistently return failure status codes even when called with valid parameters and properly initialized keys. After Investigation it was mainly do to reuse of an already declared object id, but the returned error didn't help the debugging.

RSA Key Generation Inconsistency RSA key generation using the driver function `generatePrivateRSAKey()` occasionally returns success status but fails to create a valid key object, with subsequent operations on the generated key ID returning errors. The root cause appears to be again related to object lifecycle management within the SE050.

Limited Documentation The driver includes minimal inline documentation and no comprehensive API reference. Understanding function parameter semantics, expected buffer sizes, and object lifecycle management requires examining the source code and cross-referencing with NXP SE050 documentation. This increases the learning curve for integration.

6.3.3 Missing Features

Several SE050 capabilities are not exposed by the driver:

- **ECDH Key Agreement:** The SE050 hardware supports ECDH shared secret computation through the `Se05x_API_ECDHGenerateSharedSecret` APDU command. This capability would be valuable for TLS key exchange with ephemeral keys, but it is not exposed by the current driver implementation. Adding this functionality would require implementing the appropriate APDU construction and response parsing in the driver's public API
- **Secure Object Policies:** While the SE050 supports fine-grained access policies for objects, the driver does not expose policy configuration, defaulting to unrestricted access.

6.4 Implications for PSA Integration

The analysis of the SE05X driver informed several key decisions in the PSA wrapper design:

Direct Function Mapping Given the driver’s functional API structure, the PSA wrapper must implement a direct mapping approach where each PSA function calls the corresponding SE05X driver function. For example, `psa_generate_random()` directly invokes `SE05X.random()`. This approach minimizes complexity but inherits the driver’s limitations.

Object ID Management PSA Crypto uses `psa_key_id_t` handles to reference keys. The wrapper must map these directly to SE050 object IDs using a simple integer cast (`psa_to_se_id()`). While this approach is straightforward, it assumes no conflicts between PSA key IDs and other SE050 objects and does not implement a handle translation layer.

Error Code Translation The wrapper must translates SE05X binary success/failure codes into PSA status codes (`PSA_SUCCESS`, `PSA_ERROR_HARDWARE_FAILURE`, etc.). However, due to limited error information from the driver, most hardware failures map generically to `PSA_ERROR_HARDWARE_FAILURE` without specific diagnostic information.

Feature Subset The PSA wrapper exposes only operations that are known to work reliably through the SE05X driver.

Workarounds for Missing Features For operations not directly supported by the SE05X driver, the PSA wrapper returns `PSA_ERROR_NOT_SUPPORTED`. This allows mbedTLS to fall back to software implementations transparently while still using hardware for supported operations.

6.5 Analysis Conclusions

Despite these limitations, the SE05X driver successfully demonstrates the SE050’s core capabilities and provided a functional foundation for PSA Crypto integration. The driver represents valuable prior work that significantly reduced the implementation effort for this project, and its open-source availability enables continued community improvement.

7 Project Implementation

The complete implementation is available in the project repository[1]. This section describes the design decisions and architectural components developed during the project.

7.1 System and Software Architecture

The system architecture integrates the NXP SE050 secure element with the mbedTLS library through a PSA Crypto-compatible abstraction layer. Figure 1 illustrates the layered architecture of the implementation.

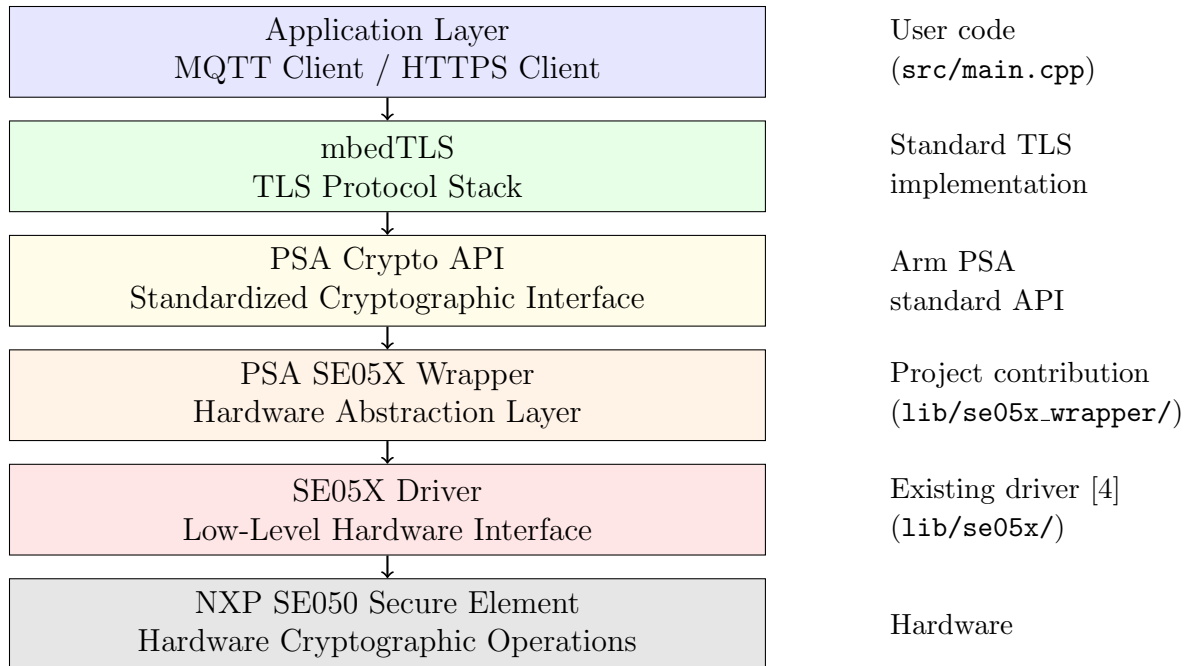


Figure 1: Layered architecture of the hardware-backed cryptographic system

The architecture follows a clear separation of concerns across six distinct layers:

- **Application Layer:** Contains user-level code such as MQTT clients or HTTPS communication examples (implementation in `src/main.cpp`, `lib/utls/mqtt.cpp`, `lib/utls/test.cpp`). This layer interacts exclusively with standard APIs and remains completely independent of the underlying hardware implementation.
- **mbedTLS Layer:** Implements the TLS protocol stack, including handshake negotiation, record layer encryption, and certificate validation. mbedTLS is configured to use PSA Crypto as its cryptographic back-end.

- **PSA Crypto API:** Provides the standardized interface defined by PSA Crypto [2]. This layer exposes cryptographic primitives such as key generation, signing, verification, hashing, and random number generation through a hardware-agnostic API.
- **PSA SE05X Wrapper:** The core contribution of this project (implementation in `lib/se05x_wrapper/`). This layer implements the PSA Crypto API by mapping PSA function calls to operations on the SE050 secure element. It translates PSA key attributes into SE050 object identifiers, ensuring that cryptographic operations are performed in hardware while maintaining API compatibility.
- **SE05X Driver:** The existing low-level driver [4] (located in `lib/se05x/`). This driver provides direct access to SE050 cryptographic functions, including ECC key generation, ECDSA signing and verification, RSA operations, AES encryption, and SHA-256 hashing.
- **NXP SE050 Hardware:** The secure element itself, which executes cryptographic operations in a tamper-resistant environment and ensures that private keys never leave the secure hardware.

This stack allows a transparent hardware offloading, enabling applications written against standard PSA Crypto or MbedTLS to automatically benefit from hardware-backed key storage and cryptographic operations without requiring any hardware-specific code modifications.

7.2 PSA Crypto Hardware Driver Design

The PSA SE05X wrapper (in `lib/se05x_wrapper/psa_se05x_wrapper.cpp` and `psa_se05x_wrapper.h`) implements a subset of the PSA Crypto API, focusing on operations directly supported by the SE050 hardware. The implementation maps PSA concepts to SE050 primitives while enforcing the security guarantees defined by the PSA specification.

7.2.1 Key Management and Opaque Keys

As stated before PSA Crypto distinguishes between *transparent keys* and *opaque keys*: While the first are accessible to the application, *opaque keys* remains hidden within secure storage. The wrapper implementation ensures that all keys generated or imported into the SE050 are treated as opaque keys.

PSA key

Each PSA key is assigned a unique `psa_key_id_t` identifier, which is directly mapped to an SE050 object identifier through a simple integer cast (see function `psa_to_se_id()` in the wrapper). The wrapper maintains this one-to-one correspondence, allowing PSA key handles to reference specific SE050 objects.

PSA key attributes

PSA key attributes, including key type, usage flags, algorithm, and lifetime, are validated during key generation and import operations. The key generation function (`psa_generate_key()`) supports both ECC P-256 and RSA 2048-bit key pairs, invoking the appropriate SE05X driver functions based on the requested key type.

7.2.2 Implemented Cryptographic Operations

The wrapper implements the following PSA Crypto operations by delegating to corresponding SE050 functions:

Random Number Generation Hardware-backed random number generation is critical for secure TLS operation. The function `psa_generate_random()` calls the SE050's true random number generator through `SE05X.random()`, ensuring that all random values used in TLS handshakes, nonces, and initialization vectors are derived from the SE050's certified entropy source rather than software-based pseudo-random number generators.

Hash Computation SHA-256 hashing is implemented in hardware through `psa_hash_compute()`, which invokes `SE05X.SHA256()`. This operation is used for TLS transcript hashing and message digest computation. Other hash methods that have a corresponding SE05X are not yet translated.

ECDSA Signing and Verification Digital signature operations are performed entirely within the SE050 through the functions `psa_sign_hash()` and `psa_verify_hash()`, ensuring that private key material never leaves the secure element. The wrapper handles both ECDSA and RSA signature algorithms, routing requests to the appropriate SE05X driver functions (`SE05X.Sign()` for ECDSA, `SE05X.SignRSA()` for RSA).

The SE050 returns signatures in DER-encoded format, which is directly compatible with TLS requirements and does not require format conversion in the wrapper layer.

Public Key Export While private keys remain opaque, public keys can be exported in DER-encoded format through `psa_export_public_key()`. This function calls `SE05X.generatePublicKey()`, which retrieves the public key component from the SE050

without exposing the private key. The exported public key can be used to construct X.509 certificates or shared with TLS peers during authentication.

AES Operations The wrapper includes support for AES-128 encryption and decryption in ECB mode through `psa_se05x_aes_encrypt()` and `psa_se05x_aes_decrypt()`. However, the SE050 does not support AES-GCM or other authenticated encryption modes required by modern TLS cipher suites, limiting the practical use of hardware-backed symmetric encryption in TLS contexts.

Mapping PSA Attributes to SE050 Objects

Table 1 summarizes the mapping between PSA Crypto attributes and SE050 object properties. The complete mapping logic and validation code can be found in the wrapper source files in the project repository.

Table 1: Mapping of PSA attributes to SE050 object properties

PSA Attribute	SE050 Property
<code>psa_key_id_t</code>	SE050 Object ID
<code>PSA_KEY_TYPE_ECC_KEY_PAIR</code>	ECC P-256 Keypair
<code>PSA_KEY_TYPE_RSA_KEY_PAIR</code>	RSA 2048-bit Keypair
<code>PSA_KEY_USAGE_SIGN_HASH</code>	Signature operation enabled
<code>PSA_KEY_USAGE_VERIFY_HASH</code>	Verification operation enabled
<code>PSA_ALG_ECDSA(SHA_256)</code>	ECDSA with SHA-256

7.3 Integration with mbedTLS

Since version 2.X MbedTLS supports PSA Crypto as a cryptographic backend through the `MBEDTLS_USE_PSA_CRYPTO` configuration option. When enabled, mbedTLS routes cryptographic operations through the PSA API rather than using its internal software implementations.

7.3.1 Configuration and Build System

The integration requires configuration of both mbedTLS and PSA Crypto. The mbedTLS configuration file (`include/mbedtls_config.h`) enables PSA support, while the PlatformIO build configuration (`platformio.ini`) ensures that the PSA wrapper library is compiled and linked with the application.

A custom RNG glue layer (`lib/mbedtls_psa_glue/mbedtls_psa_glue.cpp`) provides the bridge between mbedTLS's RNG requirements and PSA Crypto's random number generation interface. This glue function, `mbedtls_psa_get_random()`,

wraps `psa_generate_random()` and provides the function signature expected by mbedTLS configuration calls.

7.3.2 Initialization Sequence

The system initialization follows a specific sequence to ensure proper setup of all cryptographic components:

1. PSA Crypto initialization (`psa_crypto_init()`), which in turn initializes the SE050 hardware
2. Generation or loading of persistent authentication keys
3. Configuration of mbedTLS with PSA-backed RNG
4. Establishment of TCP connection
5. TLS handshake execution

This sequence is implemented in the `setup()` function in `src/main.cpp`. The initialization ensures that the SE050 is ready before any cryptographic operations are attempted, and that mbedTLS is configured to use PSA Crypto for all supported operations.

Note

For ease of use in the setup all se05x objects are deleted using the `SE05X.deleteAllObjects()`, doing this because is a test environment and during some tests i sometimes leave random objects in memory.

7.3.3 TLS Handshake and Cryptographic Operations

During a TLS handshake, mbedTLS performs several cryptographic operations. Figure 2 illustrates which operations are hardware-backed in the current implementation.

The following operations occur during a typical TLS 1.2 handshake:

- **Random number generation:** Client and server random values, nonces, and padding are generated using the PSA RNG, which delegates to the SE050 hardware. This is configured through `mbedtls_ssl_conf_rng()` in the TLS setup function (either `setup_tls()` in `lib/utls/test.cpp`, `setup_mqtt_tls()` in `lib/utls/mqtt.cpp`).
- **Key exchange:** In ECDHE cipher suites, ephemeral key pairs are generated for key agreement. The current implementation does not offload ECDH operations to hardware, as these are ephemeral keys that do not require the same level of protection as long-term authentication keys. However, the random values used for ephemeral key generation are hardware-derived.

Random Number Generation Hardware: SE050 True RNG	Hardware
ECDHE Key Exchange Software: Ephemeral keys	Software
Transcript Hashing (SHA-256) Hardware: SE050 SHA-256	Hardware
PRF / Key Derivation Software: mbedTLS	Software
AES-GCM Encryption Software: mbedTLS	Software

Figure 2: TLS handshake operations and hardware backing status

- **Hashing:** Hash operations required for transcript hashing and finished message generation can be performed using PSA Crypto, which delegates to SE050 SHA-256 hardware when `psa_hash_compute()` is called. The extent to which mbedTLS uses PSA for internal hashing depends on the specific mbedTLS version and configuration.
- **Session key derivation:** TLS derives symmetric session keys using the PRF (Pseudo-Random Function). This derivation is currently performed in software, as the SE050 does not expose HKDF or TLS-PRF primitives directly.
- **Symmetric encryption:** Record layer encryption uses AES-GCM in modern TLS implementations. The SE050 supports AES-ECB but not GCM mode, so symmetric encryption remains software-based in the current implementation.

Table 2 summarizes which TLS operations are hardware-backed in the current implementation.

One of the most critical operation (eg. random number generation) is fully hardware-backed and provides certified entropy for all the TLS operations, and private keys for persistent authentication are securely stored within the secure element.

Symmetric encryption on the other end remains mainly implemented by software due to the hardware limitations. This implementation provides a foundation for future enhancements, such as hardware-accelerated key derivation or support for alternative cipher suites that align better with SE050 capabilities.

Table 2: TLS operations and hardware backing status

Operation	Hardware	Implementation Notes
Random number generation	Yes	SE050 true RNG via <code>psa_generate_random()</code>
Key generation	Yes	ECC/RSA keys stored in SE050
Digital signing	Yes	ECDSA/RSA signing performed in SE050
Hash computation	Yes	SHA-256 operations in SE050
ECDH key exchange	No	Ephemeral keys generated in software
PRF/Key derivation	No	Software implementation (SE050 lacks HKDF support)
AES-GCM encryption	No	Software implementation (SE050 lacks GCM mode)

7.4 ECDH Key Agreement and Integration Challenges

The NXP SE050 secure element supports Elliptic Curve Diffie-Hellman (ECDH) key agreement through a dedicated APDU command [7]. In theory, this would allow the TLS handshake to perform shared secret computation directly inside the secure element, further reducing the exposure of sensitive cryptographic operations to the main microcontroller.

To explore this possibility, the existing SE05X driver was extended with a new `ECDH()` function that exposes the SE050 ECDH capability (the extended coded can be found in `lib/se05x_extended/se05x_extended.cpp`). The function accepts a private key identifier, a peer public key, and an output buffer for the generated shared secret.

Initial stand-alone tests were successful and confirmed that the SE050 can correctly compute ECDH shared secrets.

7.4.1 Integration Challenges with mbedTLS

Although the ECDH operation itself works correctly, integrating it into the mbedTLS handshake proved significantly more difficult.

During a TLS handshake using ECDHE cipher suites, mbedTLS internally performs the following operations:

1. Generation of an ephemeral ECDH key pair
2. Exchange of public keys with the peer
3. Computation of the shared secret

4. Derivation of session keys from the shared secret

These operations are tightly integrated into the internal mbedTLS handshake state machine and are not exposed through simple callback interfaces. Unlike random number generation, where mbedTLS allows the registration of a custom RNG callback, ECDH shared secret computation is performed directly inside the library internals.

As a result, replacing the software ECDH implementation with a hardware-backed version is way harder than initially expected.

7.4.2 Attempted Integration

An experimental integration attempt was performed by trying to intercept mbedTLS ECC operations and redirect them to the SE050, however, several practical problems were encountered. First, mbedTLS expects ECDH operations to operate directly on its internal ECC structures, while the SE050 performs the entire operation internally and only returns the final shared secret. This mismatch made integration difficult.

Second, TLS ephemeral keys are temporary and exist only for the duration of the handshake, while the SE050 stores objects persistently. This required generating and deleting secure element objects during every handshake, introducing additional complexity and overhead.

Due to these challenges and time limitations, full hardware-backed ECDH integration was not completed.

7.4.3 Practical Impact

The absence of hardware-backed ECDH does not prevent secure TLS communication.

In the current implementation:

- Random values used during the handshake are still generated by the SE050 TRNG
- Long-term authentication keys remain protected inside the secure element
- The ECDH shared secret computation itself is performed in software
- Session keys temporarily exist in main RAM during active TLS sessions

As a result, the system still benefits from hardware-backed entropy generation and protected key storage, even though ephemeral key exchange remains software-based.

7.5 TLS-secured MQTT Demonstration

As stated in the introduction, to validate the practical side of this integration, a complete MQTT-over-TLS demonstration was implemented. This demonstration showcases end-to-end secure communication between the Portenta C33 client and a TLS-enabled MQTT broker, with cryptographic operations transparently offloaded to the SE050 secure element where possible.

7.5.1 System Architecture

The demonstration system consists of three primary components, as illustrated in Figure 3.

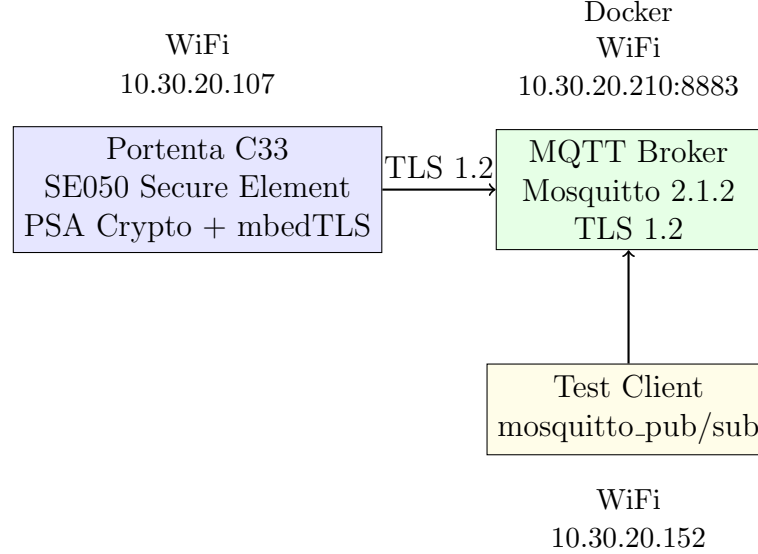


Figure 3: MQTT-over-TLS demonstration architecture

Portenta C33 Client The client device, runs the firmware described in the project and a custom MQTT-over-TLS. The implementation manually constructs MQTT protocol packets and transmits them over the TLS-secured connection established through mbedTLS. This approach was necessary because the `PubSubClient` library for Arduino does not provide native integration with custom TLS layers.

MQTT Broker The broker is deployed using Eclipse Mosquitto 2.1.2 running in a Docker container on a development workstation. The broker is configured to listen on port 8883 for TLS-secured connections and port 1883 for unencrypted connections (used during development and testing). TLS configuration includes a self-signed certificate authority, server certificate, and server private key. Client certificate authentication is disabled in the current demonstration to focus on server-side certificate verification and hardware-backed random number generation.

Test Client A standard MQTT client running on the development workstation subscribes to demonstration topics and publishes test messages, enabling bidirectional communication verification and functional testing of the complete system.

7.5.2 Certificate Infrastructure

The demonstration uses a simple public key infrastructure based on self-signed certificates generated using OpenSSL. The CA certificate is embedded in the Portenta firmware as a C header file (`include/ca_cert.h`) and loaded into mbedTLS during TLS context initialization. The server certificate is configured on the MQTT broker and presented during the TLS handshake.

7.5.3 Connection Establishment Sequence

The connection establishment sequence can be divided into eight steps, as illustrated in fig. 4. These steps span multiple layers of the communication stack, starting from network-level connectivity and progressing through secure element initialization, transport-layer setup, and cryptographic negotiation as described in fig. 5, up to application-layer protocol exchange via MQTT. Each phase is strictly ordered, as higher-layer operations depend on the successful completion of the previous layers.

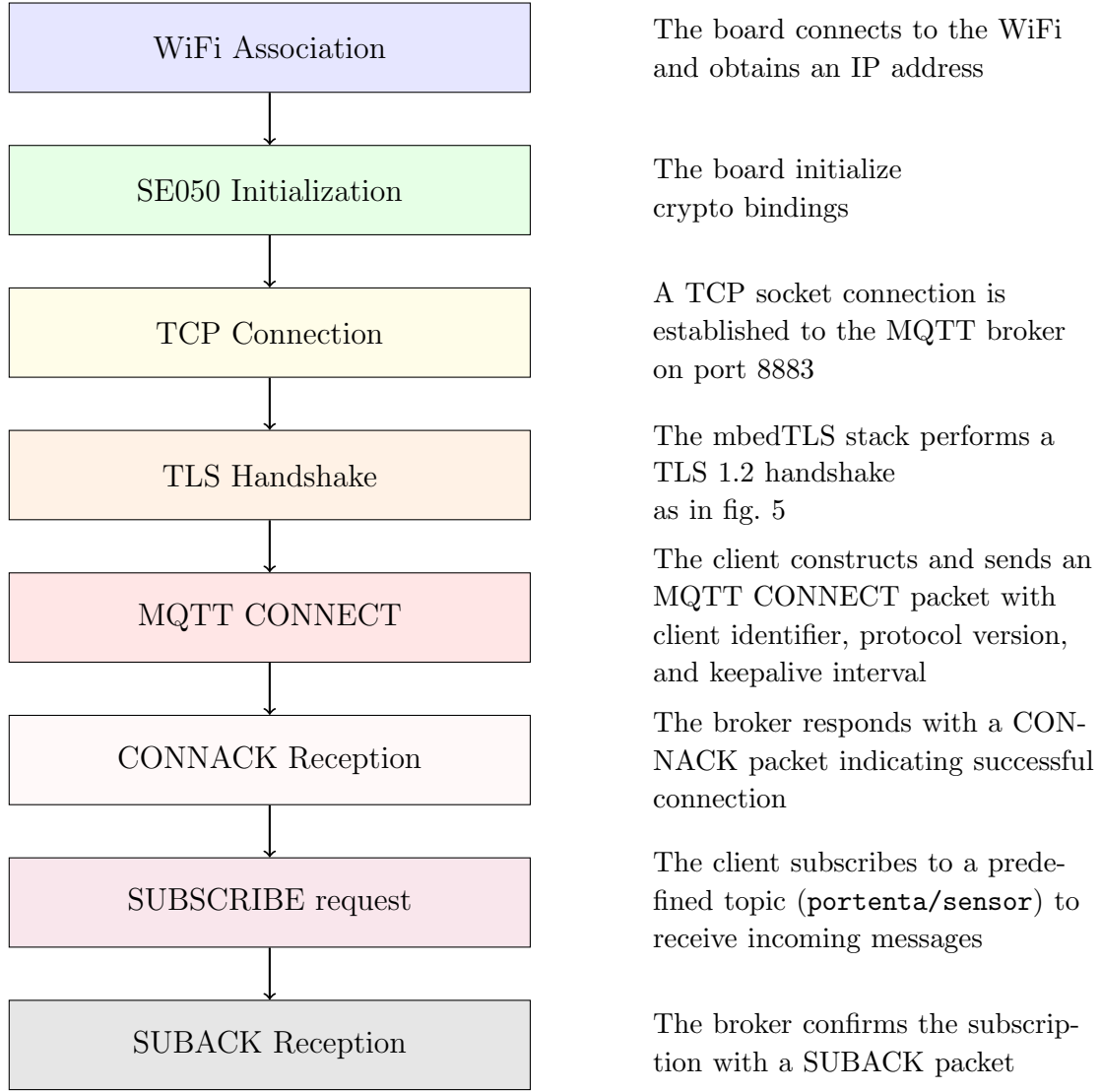


Figure 4: Complete connection establishment process

7.5.4 Message Flow and User Interaction

The demonstration provides an interactive serial console interface allowing users to publish messages to the MQTT broker. When a user types a message and presses Enter, the firmware constructs an MQTT PUBLISH packet containing the message payload, encrypts it using the established TLS session, and transmits it to the broker. The broker then distributes the message to all subscribed clients, including the PC test client, demonstrating bidirectional secure communication.

Incoming messages from other clients are received as encrypted TLS records, decrypted by mbedTLS, parsed as MQTT PUBLISH packets, and displayed on the serial

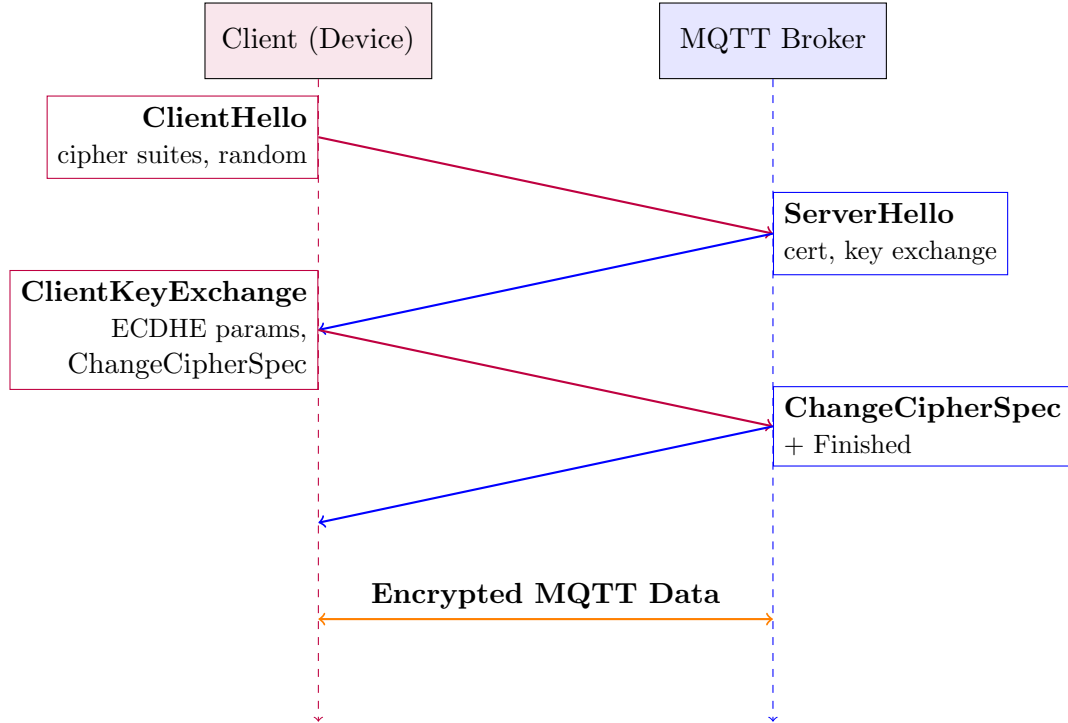


Figure 5: TLS 1.2 handshake sequence between client and MQTT broker

console with topic and payload information clearly identified.

The system maintains the MQTT connection by transmitting PINGREQ packets every 30 seconds as required by the MQTT keep-alive mechanism. These packets are automatically sent by the firmware to prevent connection timeout.

7.5.5 Hardware-Backed Security Features

Throughout the TLS session establishment and data exchange, several operations leverage the SE050 secure element. All random values required by the TLS handshake (including client random, nonces, and ephemeral key material) are generated using the SE050's TRNG, achieved through the PSA Crypto RNG interface which routes `psa_generate_random()` calls to `SE05X.random()`. Long-term authentication keys are generated within and stored inside the SE050, ensuring private key material never exists in main microcontroller memory. Additionally, if client certificate authentication were enabled, ECDSA signing operations during the TLS handshake would be performed within the SE050 using `psa_sign_hash()`, maintaining private key opacity. However, as detailed in section 8.3, symmetric encryption operations are performed in software by mbedTLS due to SE050 hardware limitations.

7.5.6 Operational Validation

The demonstration successfully validates the implementation capabilities of establishing TLS 1.2 sessions with hardware-backed random number generation, of perform MQTT-over-TLS operation and most importantly the successful integration of PSA Crypto, mbedTLS, and SE050 in a real-world IoT communication scenario.

8 Project Evaluation

This project successfully demonstrates the integration of the NXP SE050 secure element with PSA Crypto and mbedTLS on the Arduino Portenta C33 platform. The implementation validates that hardware-backed cryptographic operations can be integrated into a standard TLS workflow while preserving compatibility with existing APIs and software stacks.

The developed PSA wrapper allows applications to use the SE050 through standard PSA Crypto interfaces without requiring direct interaction with the hardware-specific SE05X driver. This abstraction simplifies application development and demonstrates the practical value of standardized cryptographic APIs in embedded systems.

8.1 Achieved Objectives

The main project objectives were successfully completed:

- A PSA-compatible wrapper layer was implemented over the existing SE05X driver.
- Hardware-backed random number generation was integrated into mbedTLS through PSA Crypto.
- ECC and RSA key generation, signing, verification, and hashing operations were successfully mapped to the SE050 secure element.
- Private keys remain securely stored inside the SE050 and are never exposed in application memory.
- A working MQTT-over-TLS demonstration was implemented and validated using the Portenta C33 and a Mosquitto broker.
- TLS-secured communication was successfully established using mbedTLS integrated with PSA Crypto.

The implementation confirms that the SE050 can be used as a practical hardware-backed cryptographic provider for embedded TLS applications.

8.2 Performance and Practical Observations

The integration process showed that hardware-backed cryptography introduces additional implementation complexity compared to pure software solutions. In particular, object management inside the SE050 requires careful handling of object identifiers and lifecycle management.

Despite this complexity, the system demonstrated robust stability during extended testing. A continuous 4-hour test was conducted in which the MQTT-over-TLS client

published a message every 30 seconds (see `stability_test.sh`), resulting in 481 successfully transmitted and received messages without any disconnection or cryptographic error (see `logs/board.log` and `logs/script.log`). The system also successfully established TLS connections, and random number generation as well as signing operations worked reliably through the PSA abstraction layer.

The use of PlatformIO and the Arduino framework proved sufficient for demonstrating secure communication without requiring a full RTOS environment such as Zephyr or Mbed OS.

From a usability perspective, PSA Crypto significantly simplifies the interaction between applications and cryptographic hardware. Once integrated, the application layer does not need to directly manage SE050-specific operations.

8.3 Limitations and Open Issues

Although the project successfully demonstrates hardware-backed cryptographic integration, several limitations remain.

8.3.1 SE050 Hardware Constraints

As previously discussed in section 4.3, the Portenta C33 integrates the SE050C2 variant, which does not support AES-GCM or other AEAD cipher modes required by modern TLS implementations.

As a result, symmetric TLS encryption and decryption are still performed in software by mbedTLS. While hardware-generated random numbers and protected key storage improve security, TLS session keys still temporarily exist in main RAM during active sessions.

8.3.2 Limited Hardware Offloading

Only a subset of TLS cryptographic operations could be transparently offloaded to the SE050. Operations such as:

- ECDHE shared secret computation
- TLS PRF and key derivation
- AES-GCM encryption

remain software-based due to either hardware limitations or integration complexity.

In particular, integrating hardware-backed ECDH into the mbedTLS handshake proved difficult because the library does not expose clean callback interfaces for elliptic curve key exchange operations.

8.3.3 Driver Maturity

The existing SE05X driver provided a solid foundation for the project, but several operational limitations were encountered during development:

- Limited error reporting
- Incomplete AES mode support
- Minimal documentation

These limitations increased debugging complexity and required additional testing to identify hardware and object management issues.

8.4 Security Considerations

Even with the limitations described above, the implementation improves the security model compared to software-only TLS solutions.

The most important security improvement is that long-term private keys remain inside the SE050 secure element and are never directly accessible by the application or stored in external memory.

Additionally, the SE050 hardware random number generator provides high-quality entropy for TLS operations, reducing the risk of weak random generation on embedded platforms.

However, because symmetric session keys and ephemeral ECDH computations remain software-based, the implementation cannot be considered a fully hardware-backed TLS solution. Instead, it represents a hybrid architecture where the most sensitive long-term secrets can be hardware protected while some temporary session material remains software managed.

8.5 Future Work

Several improvements could extend the current implementation:

- Full integration of hardware-backed ECDH operations into the TLS handshake
- Support for additional PSA Crypto operations and algorithms
- Improved SE05X driver error handling and documentation
- Evaluation of newer SE050 variants with AES-GCM support
- Integration of client certificate authentication using hardware-protected keys

These improvements would move the system closer to a fully hardware-backed TLS implementation more suitable for real world IoT deployments.

9 Conclusion

This project demonstrated the integration of the NXP SE050 secure element with PSA Crypto and mbedTLS on the Arduino Portenta C33 platform. The main objective was to create a standardized hardware-backed cryptographic layer for secure TLS communication in embedded and IoT systems.

A PSA-compatible wrapper was implemented over the existing SE05X driver, allowing applications to use the SE050 through standard PSA Crypto APIs.

The MQTT-over-TLS demonstration confirmed that secure TLS communication can be established while using the SE050 for some important cryptographic operations.

During development, several limitations were encountered. The SE050C2 does not support AES-GCM, which means TLS symmetric encryption is still performed in software. In addition, integrating hardware-backed ECDH into mbedTLS proved difficult due to the internal structure of the TLS handshake implementation.

Despite these limitations, the project shows that PSA Crypto provides a useful abstraction layer for integrating secure elements into embedded TLS applications. The implementation also provides a foundation for future improvements such as hardware-backed ECDH support and more complete TLS offloading.

9.1 Social, Ethical, and Legal Aspects

This project focuses on improving cryptographic security in embedded and IoT systems through the integration of a hardware secure element with PSA Crypto and mbedTLS. From a social perspective, stronger device-level security contributes to increased trust in IoT ecosystems, which are often perceived as vulnerable due to resource constraints and inconsistent security implementations.

Ethically, the use of hardware-backed cryptography supports the principle of data protection by design, reducing the risk of sensitive information exposure in transit or at rest. However, it also raises considerations regarding responsible implementation: secure elements may be used to protect legitimate communications, but the same mechanisms could theoretically be embedded in opaque systems that limit auditability or transparency if misused.

From a legal standpoint, the project aligns with modern cybersecurity and data protection regulations, which require appropriate technical measures to ensure data confidentiality and integrity. The use of standardized cryptographic primitives and secure hardware helps support compliance with these requirements. At the same time, any deployment of such systems in real-world products must consider export control regulations on cryptographic technologies and ensure proper handling of cryptographic keys and sensitive data throughout the device lifecycle.

References

- [1] De Biasi Alessandro. *SE05X-psa-tls GitHub Page*. URL: <https://github.com/lilvirgola/SE05X-psa-tls>.
- [2] *Arm PSA Certified Crypto API Specification*. Arm Ltd. URL: <https://arm-software.github.io/psa-api/crypto/>.
- [3] Luigi Atzori, Antonio Iera, and Giacomo Morabito. “The Internet of Things: A survey”. In: *Computer Networks* 54.15 (2010), pp. 2787–2805. ISSN: 1389-1286. DOI: <https://doi.org/10.1016/j.comnet.2010.05.010>. URL: <https://www.sciencedirect.com/science/article/pii/S1389128610001568>.
- [4] Matthias Kuschnig. *SE05X Driver GitHub Page*. URL: https://github.com/matthias-ku/SE05X_Final.
- [5] *mbed TLS Documentation*. Arm Ltd. URL: <https://mbed-tls.readthedocs.io/en/latest/>.
- [6] *MQTT Version 5.0*. OASIS Standard. URL: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [7] *SE050 Plug & Trust Secure Element Product Data Sheet*. NXP Semiconductors. URL: <https://www.nxp.com/docs/en/data-sheet/SE050-DATASHEET.pdf>.
- [8] *wolfSSL Embedded SSL/TLS Library*. wolfSSL Inc. URL: <https://www.wolfssl.com>.